実装計画: 電力需要予測システム GitHub Pages版

ブランチ: `feature/impl-001-Power-Demand-Forecast` | **作成日**: 2025-11-26 | **仕様**: spec.md **入力**: 機能仕様書 `/specs/001-Power-Demand-Forecast/spec.md`

備考: この計画書は`/speckit.plan`コマンドにより機能仕様書に基づいて生成されました。

概要

電力需要予測システムのGitHub Pages化と統合ダッシュボード構築。ブラウザ操作のみで4つの機械学習モデル（LightGBM、Keras、RandomForest、PyCaret）を選択・学習・予測できるWebインターフェースを提供する。ローリング時系列交差検証による学習年最適化、Open-Meteo APIによる気温データ取得、GitHub Actionsによる毎日自動実行を実装。

技術コンテキスト

言語・バージョン: Python 3.10.11（固定バージョン） **主要依存関係**:

- LightGBM 3.3.5（勾配ブースティング）
- Keras/TensorFlow 2.13.0（深層学習）
- scikit-learn 1.3.0（RandomForest）
- PyCaret 3.0.4（AutoML）
- pandas 2.0.3（データ処理）
- Flask/http.server（HTTPサーバー）

ストレージ: CSVファイルベース（電力需要・気温データ）、学習済みモデル（.sav/.h5）、localStorage（ブラウザ側学習年永続化） **テスト**: pytest（単体テスト）、GitHub Actions（統合テスト）、R^2スコア閾値テスト（> 0.80） **対象プラットフォーム**: GitHub Pages（静的ホスティング）、GitHub Actions（毎日JST 07:00自動実行）、ローカルHTTPサーバー（Python） **プロジェクトタイプ**: Web（フロントエンド: HTML/CSS/JavaScript、バックエンド: Python HTTPサーバー） **パフォーマンス目標**:

- LightGBM学習時間 < 30秒（GitHub Actions）
- Keras学習時間 < 60秒（GitHub Actions）
- ダッシュボードAPI応答時間 < 2秒（ローカル）
- R^2スコア > 0.80（全モデル）

制約:

- GitHub Actions無料枠: 月2000分実行時間制限
- Open-Meteo API無料版: 1時間10000リクエスト制限
- localStorage容量: 5MB制限（学習年選択のみ保存のため問題なし）
- GitHub Pages: 静的ファイルのみ（動的バックエンド不可）

規模・範囲:

- データ量: 2016-2025年の10年間電力需要・気温データ（約87,600レコード）
- モデル数: 4種類（LightGBM、Keras、RandomForest、PyCaret）
- ユーザー数: 開発者・研究者向け個人利用規模
- 予測頻度: 毎日1回明日24時間予測

憲法準拠チェック

ゲート: Phase 0（研究）開始前に合格必須。Phase 1（設計）完了後に再チェック。

原則	要件	ステータス	備考
I. TDD徹底	テスト先行実装、R^2 > 0.80の閾値テスト必須	✅ PASS	GitHub Actionsで毎日精度テスト実行、閾値未達でIssue自動作成
II. セキュリティファースト	外部API通信HTTPS必須、機密データ平文保存禁止	✅ PASS	Open-Meteo APIはHTTPS通信、localStorage は学習年のみ（個人情報なし）
III. パフォーマンス定量化	LightGBM < 30秒、Keras < 60秒、API < 2秒、R^2 > 0.80	✅ PASS	spec.mdで成功基準として明記済み
IV. データ保護	機密データ暗号化、公開データのみ使用	✅ PASS	公開気温・電力需要データのみ、個人情報なし
V. 依存関係再現性	requirements.txt でバージョン固定、Python 3.10.11明記	✅ PASS	requirements.txtで==固定、Python 3.10.11指定
VI. 仕様実装乖離検出	仕様書とコードの整合性検証、ドキュメント同期更新	✅ PASS	spec.md、plan.md、README、DEPLOY_GUIDEの整合性確保

総合評価: ✅ **GATE PASS** - すべての憲法原則に準拠しています。

Phase 0完了: research.md生成完了 **Phase 1完了**: data-model.md、contracts/api-spec.yaml、quickstart.md生成完了

Phase 1後の再評価:

原則	Phase 1後の検証	ステータス
I. TDD徹底	API契約テスト（OpenAPI仕様）定義済み	✅ PASS
II. セキュリティファースト	HTTPS通信、localStorage学習年のみ確認済み	✅ PASS
III. パフォーマンス定量化	data-model.mdでメモリ最適化（float32）定義	✅ PASS
IV. データ保護	公開データのみ、個人情報なし確認済み	✅ PASS
V. 依存関係再現性	requirements.txt参照、バージョン固定確認済み	✅ PASS
VI. 仕様実装乖離検出	plan.md、research.md、data-model.md、quickstart.mdの整合性確保	✅ PASS

Phase 1総合評価: ✅ **GATE PASS** - Phase 1完了後も憲法準拠を維持しています。

次のステップ: Phase 2（タスク分解）は`/speckit.tasks`コマンドで実行します（このコマンドでは作成されません）。

プロジェクト構造

ドキュメント（本機能）

```
specs/feature/impl-001-Power-Demand-Forecast/
├── plan.md              # この計画書（/speckit.planコマンド出力）
├── research.md          # Phase 0出力（/speckit.planコマンド）
├── data-model.md        # Phase 1出力（/speckit.planコマンド）
├── quickstart.md        # Phase 1出力（/speckit.planコマンド）
├── contracts/           # Phase 1出力（/speckit.planコマンド）
└── tasks.md             # Phase 2出力（/speckit.tasksコマンド - このコマンドでは作成されない）
```

ソースコード（リポジトリルート）

```
# Webアプリケーション（フロントエンド + バックエンド）

Power-Demand-Forecast/
├── AI/                              # バックエンド（Python HTTPサーバー + 機械学習）
│   ├── server.py                    # HTTPサーバー（Port 8002）
│   ├── calculate_metrics.py         # メトリクス計算
│   ├── generate_metrics.py          # メトリクス生成
│   ├── metrics.json                 # メトリクスデータ
│   ├── requirements.txt             # Python依存パッケージ（バージョン固定）
│   ├── dashboard/                   # フロントエンド（HTML/CSS/JavaScript）
│   │   ├── index.html               # ダッシュボードUI
│   │   └── localStorage_test.html   # localStorage動作確認
│   ├── data/                        # データ処理
│   │   ├── data.py                  # データ統合処理
│   │   ├── juyo-YYYY.csv            # 電力需要データ（2016-2025年）
│   │   ├── temperature-YYYY.csv     # 気温データ（2016-2024年）
│   │   ├── X.csv                    # 特徴量データ
│   │   ├── Y.csv                    # ラベルデータ
│   │   ├── Xtrain.csv               # 訓練用特徴量
│   │   ├── Ytrain.csv               # 訓練用ラベル
│   │   ├── Xtest.csv                # テスト用特徴量
│   │   └── Ytest.csv                # テスト用ラベル
│   ├── train/                       # モデル学習
│   │   ├── LightGBM/                # LightGBMモデル
│   │   │   ├── LightGBM_train.py    # 学習スクリプト
│   │   │   ├── LightGBM_optimize_years.py  # 学習年最適化
│   │   │   ├── LightGBM_model.sav   # 学習済みモデル
│   │   │   ├── LightGBM_Ypred.csv   # 予測結果
│   │   │   └── YYYY-MM-DD_LightGBM_optimize_years.txt  # 最適化ログ
│   │   ├── Keras/                   # Kerasモデル（同構成）
│   │   ├── RandomForest/            # RandomForestモデル（同構成）
│   │   └── Pycaret/                 # PyCaretモデル（同構成）
│   └── tomorrow/                    # 明日予測
│       ├── data.py                  # 予測用データ処理
│       ├── temp.py                  # Open-Meteo API気温取得
│       ├── tomorrow.csv             # 明日気温データ
│       ├── Ytest.csv                # テストラベル
│       ├── LightGBM/                # LightGBM明日予測
│       │   ├── LightGBM_tomorrow.py
│       │   └── LightGBM_tomorrow.csv
│       ├── Keras/                   # Keras明日予測（同構成）
│       ├── RandomForest/            # RandomForest明日予測（同構成）
│       └── Pycaret/                 # PyCaret明日予測（同構成）
├── docs/                            # ドキュメント
│   ├── 完全仕様書.md               # 詳細仕様書
│   ├── 使用手順書.md               # ローカル環境セットアップ
│   └── DEPLOY_GUIDE.md              # GitHub Actionsデプロイガイド
├── .github/                         # GitHub Actions
│   └── workflows/
│       └── daily-forecast.yml       # 毎日JST 07:00自動実行
├── start-dashboard.ps1              # ワンコマンド起動スクリプト
├── index.html                       # GitHub Pagesトップページ
└── README.md                        # プロジェクト概要
```

構造決定:

このプロジェクトはWebアプリケーション構成を採用しています。バックエンドはAI/ディレクトリにPython HTTPサーバーと機械学習パイプラインを配置し、フロントエンドはAI/dashboard/ディレクトリに静的HTML/CSS/JavaScriptを配置しています。

選択理由：

1. **明確な責任分離**: フロントエンド（ダッシュボードUI）とバックエンド（機械学習処理）が物理的に分離
2. **既存構造の尊重**: 既存のAI/ディレクトリ構造を維持し、互換性を確保
3. **GitHub Pages対応**: 静的ファイル（index.html、予測PNG）をGitHub Pagesで公開可能
4. **ローカル開発の容易性**: start-dashboard.ps1でワンコマンド起動

複雑性追跡

> ***憲法違反がある場合のみ記入***

憲法違反なし - このセクションは空白です。